

TrialCheck

TrialCheck / Interview Defense Document

Sidharth Kriplani

Document v1.0 · Sourced exclusively from repo files

Label	Meaning
[IMPLEMENTED]	Code exists, runs, produces verified artifact or DB row
[SIMULATED]	Deliberately synthetic scenario; simulation logic is real code
[PARTIALLY VERIFIED]	Code exists but verification relies on internal consistency only
[FUTURE]	Not built — explicitly absent from the repo

TrialCheck Interview Defense Document v2.0

Complete Edition — A/B Experiment Readout Auditing

SECTION 1 — PROJECT IDENTITY

1.1 One-Sentence Summary

TrialCheck audits completed A/B experiment readouts by running seven structured checks — SRM, pre-period balance, primary metric significance, continuous metric significance, peeking risk, practical significance, and guardrail movement — and returns a structured PASS/WARN/FAIL report the team reviews before making a ship/no-ship decision.

1.2 The Problem This Solves

The problem is not that teams don't know what to check before shipping an experiment. Senior data scientists have a consistent mental checklist: Did assignment work? Was this called early? Is the lift real and large enough to matter? Did any guardrail move in the wrong direction? Were groups balanced before the test started? The problem is that this checklist lives in runbooks, reviewer notes, and institutional memory — applied inconsistently, skipped under deadline pressure, and invisible to junior practitioners who haven't learned it yet.

A fast-moving team will ship on a p-value alone. A careful one catches the SRM at review, the peeking three weeks later, and the guardrail regression only in production. TrialCheck packages that senior DS checklist into a library call.

1.3 Claim Boundary

TrialCheck is an audit helper, not a decision-maker. It does not run experiments, assign users, replace an experimentation platform, prove causal validity, perform CUPED or sequential testing, correct for multiple comparisons across metrics, or make automatic ship/no-ship decisions. These limitations are explicitly stated in every report output.

SECTION 2 — THE SEVEN CHECKS

2.1 Check Architecture

Group	Check	Method	Fires When
Randomization	SRM	chi ² df=1, exact p via erfc	Observed split deviates from planned ratio
Randomization	Pre-period balance	SMD per covariate, pooled SD	Any covariate SMD ≥ 0.1
Statistical	Primary metric	Two-proportion z-test, pooled SE	p > alpha or missing counts
Statistical	Continuous metric	Welch t-test, Welch–Satterthwaite df	p > alpha for mean comparison
Statistical	Peeking risk	Duration ratio + interim look count	Test called before planned duration
Practical	Practical significance	Observed lift vs business threshold	Lift real but negligible
Practical	Guardrail movement	Direction check + tolerance band	Guardrail moves in bad direction

2.2 Sample Ratio Mismatch (SRM) [IMPLEMENTED]

SRM occurs when the actual assignment ratio differs from the planned ratio. Planned 50/50 split but observed 53/47: that 6-point difference might be random noise or might indicate assignment bias. TrialCheck computes the chi-squared statistic for df=1 using the exact formula: $\chi^2 = (n_{\text{control}} -$

$$\frac{(n_expected_control)^2}{n_expected_control + (n_treatment - n_expected_treatment)^2} \cdot \frac{(n_treatment - n_expected_treatment)^2}{n_expected_treatment}$$
, then converts to a p-value using $\text{erfc}(\sqrt{\chi^2/2})$.

SRM is a FAIL-level finding because it invalidates the randomization assumption that every other statistical test depends on. An SRM result means: the groups being compared are not randomly sampled from the same population, so any observed metric differences may be assignment artifacts rather than treatment effects.

2.3 Pre-Period Balance [IMPLEMENTED]

Even with correct assignment ratios, the two groups might have different pre-experiment covariate distributions by chance (especially in small experiments). The check computes Standardised Mean Difference (SMD) for each provided covariate: $\text{SMD} = (\text{mean_treatment} - \text{mean_control}) / \text{pooled_std}$. An $\text{SMD} \geq 0.1$ on any covariate is flagged WARN — it indicates the groups differed before the treatment began, confounding the observed metric effects.

Pre-period balance doesn't fire FAIL because imbalance is often addressable by CUPED (covariate adjustment) rather than requiring re-randomization.

2.4 Primary Metric Significance [IMPLEMENTED]

The primary metric check runs a two-proportion z-test for binary outcomes (conversion rate, click rate, activation rate). The test statistic is: $z = (p_t - p_c) / \text{SE_pooled}$, where $\text{SE_pooled} = \sqrt{(p(1 - p) \times (1/n_t + 1/n_c))}$ and p is the pooled conversion rate. Statistical significance is assessed at the configured α level (default: 0.05).

A non-significant primary metric result is WARN (not FAIL) by default — statistical non-significance doesn't mean the experiment failed, it might mean it was underpowered. TrialCheck documents this distinction: "not significant at $\alpha=0.05$ " versus "no lift detected."

2.5 Continuous Metric Significance [IMPLEMENTED]

For continuous outcome metrics (revenue per user, session duration, order value), the check uses a Welch t-test rather than a Student's t-test. Welch's t-test does not assume equal variances between

groups, which is almost always the correct assumption for real A/B test data (treatment group variances differ from control due to the treatment effect itself).

The Welch–Satterthwaite equation computes the effective degrees of freedom: $df = (s_t^2/n_t + s_c^2/n_c)^2 / ((s_t^2/n_t)^2/(n_t-1) + (s_c^2/n_c)^2/(n_c-1))$. This produces a more conservative (larger) p-value than Student's t-test when variances are unequal, reducing false positives.

2.6 Peeking Risk [IMPLEMENTED]

Peeking is calling an experiment early when results look significant, then continuing if they don't. The inflation effect is severe: peeking at $\alpha=0.05$ in the middle of an experiment raises the effective false positive rate to ~8-14% depending on how many interim looks occur. TrialCheck checks two signals: (1) `actual_duration / planned_duration < 0.8` — the test was called before 80% of the planned duration, and (2) `interim_looks ≥ 3` with no sequential testing correction applied. Either condition produces WARN with the peeking risk level.

2.7 Practical Significance [IMPLEMENTED]

Statistical significance answers "is the lift non-zero?" Practical significance answers "is the lift large enough to matter?" A treatment that increases conversion from 3.000% to 3.001% may be statistically significant at $p=0.001$ (with large enough sample) but is practically meaningless for most products. TrialCheck computes whether `observed_lift ≥ practical_significance_threshold` (configurable, default: 1% relative lift). Experiments that are statistically significant but practically negligible are flagged WARN with the observed lift and threshold shown.

This check is explicitly about business impact, not statistical validity — it is documented as decision support for the ship/no-ship call, not a statistical test.

2.8 Guardrail Movement [IMPLEMENTED]

Guardrail metrics are metrics the experiment must not harm — latency, error rate, revenue per user (in engagement experiments), or safety metrics. For each guardrail metric provided, TrialCheck checks: (1) did the metric move in the bad direction (e.g., latency increased), and (2) did the movement exceed the `guardrail_tolerance` (default: 1% relative). Both conditions must hold for a WARN to fire.

The tolerance band prevents noise from triggering WARN: a 0.1% latency increase in a large experiment is likely measurement noise; a 2% latency increase is a genuine degradation.

SECTION 3 — API AND USAGE

3.1 Core API

```
from trialcheck import TrialChecker, ExperimentSummary, GuardrailMetric
experiment = ExperimentSummary( n_control=50120, n_treatment=49880, planned_split=0.5, control_conversions=1500,
treatment_conversions=1620, alpha=0.05, planned_duration_days=14, actual_duration_days=11,
guardrails=[ GuardrailMetric("p99_latency_ms", control_val=234.1, treatment_val=241.7,
direction="lower_is_better", tolerance=0.02), ], )
checker = TrialChecker(experiment)
report = checker.audit()
print(report.overall_status) # FAIL / WARN / PASS
print(report.to_markdown())
```

3.2 v0.3.0 New Checks: Min Sample Size + Novelty Effect [IMPLEMENTED]

v0.3.0 added two new checks:

Minimum sample size: Computes the required sample size for the observed effect size at the configured power (default: 0.80) and alpha. If the actual sample is below the required sample, the experiment is flagged WARN for underpowering — the observed lift may be noise.

Novelty effect: Flags experiments where (1) the treatment shows a large lift in the first 30% of the experiment window that (2) declines toward the end. This pattern is consistent with novelty-driven behavior change that will not persist post-launch. Requires `time_series_data` to be provided.

SECTION 4 — DESIGN DECISIONS

4.1 Zero Dependencies

TrialCheck has no external dependencies. All statistical tests are implemented from scratch using Python's `math` module (`erfc`, `sqrt`, `exp`) and the standard library. This is a deliberate design decision: ML teams cannot always control their dependency environment, and a tool that requires `scipy`, `statsmodels`, or `numpy` can fail to install in constrained environments. Zero dependencies mean TrialCheck works in any Python 3.10+ environment.

The implementation correctness is validated against `scipy` reference values in the test suite: `scipy.stats.chi2_contingency`, `scipy.stats.ttest_ind`, `scipy.stats.norm` are used as ground truth in tests but not as runtime dependencies.

4.2 Platform-Agnostic Design

TrialCheck accepts a plain `ExperimentSummary` object — a simple dataclass with numbers, not platform-specific objects. This means it works regardless of whether experiments are run on Optimizely, Split.io, an internal platform, or a hand-rolled randomization system. The input schema is the smallest set of numbers needed to run the checks: assignment counts, metric outcomes, optional guardrails and covariates.

4.3 FAIL Reserved for SRM Only

SRM is the only FAIL-level finding because it is the only condition that invalidates all other checks. An experiment with SRM cannot be salvaged: the groups being compared are not comparable, so no amount of statistical adjustment produces trustworthy results. Every other finding — peeking, practical insignificance, guardrail movement — produces WARN because each is individually addressable or contextually interpretable.

SECTION 5 — Q&A

Q: Why Welch's t-test instead of Student's t-test for continuous metrics?

Student's t-test assumes equal variances between the two groups (homoscedasticity). In A/B tests, this assumption is almost always violated: the treatment group's metric variance is affected by the treatment

itself. For revenue experiments, high-spending users in the treatment group create a longer right tail, increasing variance. For latency experiments, the treatment might affect p99 latency differently from mean latency, changing the variance structure. Welch's t-test makes no homoscedasticity assumption and is uniformly more appropriate for A/B test data. The cost is a slightly more complex degrees-of-freedom computation (Welch–Satterthwaite), which TrialCheck handles internally.

Q: What is SMD and why 0.1 as the threshold for pre-period balance?

Standardised Mean Difference (SMD) = $(\text{mean_A} - \text{mean_B}) / \text{pooled_std}$. It measures the difference between groups in standard deviation units, making it comparable across metrics with different scales (age in years vs. revenue in dollars both produce SMD in SD units). The 0.1 threshold corresponds to a "small" effect size by Cohen's conventions (small = 0.2, but 0.1 is used for pre-period because even small pre-existing differences compound with the treatment effect).

Q: How does TrialCheck handle multi-metric experiments?

TrialCheck processes guardrail metrics independently (each gets its own WARN/PASS). For primary and secondary metrics, each metric is checked independently. Multiple comparison correction is not applied automatically — this is documented as a known limitation. The rationale: applying Bonferroni correction to the primary metric check is overly conservative (you specified one primary metric; adjusting for additional secondary metrics is a separate decision). Teams that want multiple comparison correction should manually adjust their alpha before passing it to TrialCheck.

Q: When is the peeking risk check most important?

The peeking risk check is most important in organizations with real-time dashboards showing experiment results. Real-time dashboards create systematic peeking: anyone with access to the dashboard can see live results and be tempted (or pressured) to call the experiment early when it "looks good." TrialCheck's peeking check doesn't prevent access to dashboards — it documents that an experiment was called early and quantifies the risk in the formal readout, creating an auditable record.

Q: What's the difference between TrialCheck and MetaSignal?

TrialCheck is a single-experiment readout auditor: it takes one experiment's results and checks their validity. MetaSignal is a platform: it includes CUPED variance reduction, A/A calibration (1,000-run simulation to verify false positive rate), sequential testing (mSPRT), and guardrail-first decisioning at the platform level. TrialCheck is the tool you use when you have an experiment result and need to audit it before shipping; MetaSignal is the infrastructure that generates those results with proper statistical safeguards built in.

SECTION 6 — NUMBERS AT A GLANCE

Metric	Value
Checks	7 (v0.2.0) + 2 new in v0.3.0 = 9 total
Checks that can FAIL	1 (SRM only)
Checks that WARN	8
Current version	v0.3.0
Test cases	17
Demo scenarios	4
Dependencies	0 (pure Python stdlib)
Output formats	JSON, Markdown, HTML

TrialCheck Interview Defense Document v2.0 — Complete Edition

Seven Checks · SRM FAIL · Welch t-test · Peeking Risk · Zero Dependencies

SECTION 7 — DEMO SCENARIOS

7.1 Four Built-In Demo Scenarios

TrialCheck ships with four built-in demo scenarios that exercise distinct check combinations:

Scenario 1 — Clean experiment: All 9 checks pass. Designed to show what a healthy readout looks like. Planned 50/50 split, actual 50005/49995, 14-day duration, primary metric $p=0.031$, lift=2.3%, no guardrail movement. Overall: PASS.

Scenario 2 — SRM failure: Planned 50/50 split, actual 62/38. Chi-squared statistic = 47.8, $p < 0.001$. All other checks also have suspicious results because the groups aren't comparable. Overall: FAIL.

Scenario 3 — Peeking + practical insignificance: 7-day actual duration against 14-day planned, 3 interim looks. Primary metric is statistically significant ($p=0.018$) but lift=0.3%, below the 1% practical threshold. Overall: WARN (two findings).

Scenario 4 — Guardrail regression: Primary metric shows a significant positive lift ($p=0.004$, lift=4.1%). But p99 latency increased by 3.2%, above the 2% guardrail tolerance. Classic "feature improves conversion but breaks performance" scenario. Overall: WARN with explicit guardrail call-out.

These four scenarios cover the complete decision space: shipping a good result, blocking a broken result, holding for underpowering/peeking, and flagging a guardrail tradeoff for stakeholder review.

7.2 Claim Boundary Reminder

TrialCheck surfaces risks. It does not make ship decisions. Scenario 4's WARN requires a product and engineering conversation: is a 3.2% latency increase acceptable for a 4.1% conversion lift? TrialCheck documents that the tradeoff exists — the decision is human.

TrialCheck Interview Defense Document v2.0 — Complete Edition

Seven Checks · SRM FAIL · Welch t-test · Peeking Risk · Zero Dependencies · Four Demo Scenarios